

2.2 Typumwandlung

Yusuf hat den Code rechts entworfen. Er führt das Programm aus und erhält die rechts stehenden Bildschirmausgaben.

```
println(3 + 5);           Ausgabe → 8
println("3" + "5");       → ?

println("Test" + 5 + 8);   → ?
println("Test" + (5 + 8)); → Test13
```

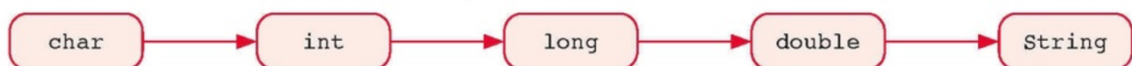
Erkläre wie die Ausgaben zustande kommen.

Aufgabe 1)

- Öffne die Online-IDE und öffne das Projekt 10 2-2 Intro. Führe das Programm aus und überprüfe deine Vermutungen.
- Ergänze weitere Kombinationen der vierten Zeile. (TODO1)

Implizite Typumwandlung

Bei **1b)** findet eine so genannte implizite _____ statt, da die Zahl 5, welche üblicherweise vom Datentyp `int` ist, als _____ (Datentyp von Text) interpretiert wird und somit das Plus-Zeichen nicht als Rechenoperator, sondern als Verkettung (Aneinanderhängen) der Zeichen genutzt wird. Die implizite Typumwandlung ist in Java nur möglich, wenn dadurch _____ Informationen verloren gehen. Daher gilt in Java folgende Kette:



Jeder Datentyp kann direkt in den weiter rechts stehenden Datentyp umgewandelt werden. Ein `int` wird also z. B. ohne Umweg in einen `String` umgewandelt.

- Betrachte die Zeilen nach den `System.exit(1)` statements. Überlege zunächst wieder was du erwarten würdest und entferne dann die mit TODOs versehenen Zeilen. Erkläre wie die Ergebnisse zustande kommen (lies weiter unten).

Explizite Typumwandlung

Die explizite Typumwandlung wird explizit als Anweisung im Programmtext angegeben. Hierfür muss man den gewünschten Zieldatentyp in runden Klammern vor die umzuwandelnde Variable schreiben:

```
int i = (int) 3.7;      char c = (char) 65;      i = (int) 4294967298;
```

Typumwandlung	Startwert	Endwert
double → int	3.0	3
	3.5	3
int → char	65	'A'
	9651	'△'
long → int	300	300
	4294967298	2

Achtung: Hierbei kann es zu _____ (-> nicht alles bleibt erhalten) kommen.

2.3 Polymorphismus

Beschreibe das Klassendiagramm rechts und erkläre, warum die folgenden Aufrufe möglich sind. Achte auf die Vererbungs-Beziehungen (Ober- und Unterklassen).

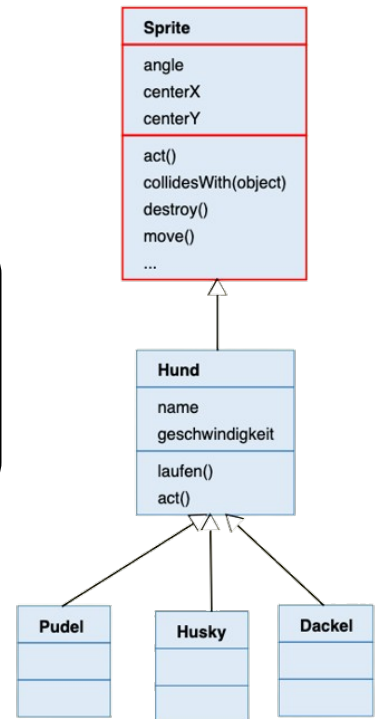
```
5 // Hunde
6 Hund hund1 = new Pudel();
7 Hund hund2 = new Dackel();
8 Hund hund3 = new Husky();
```

Aufgabe 2) 10_2-3_Hunde

- Öffne das Projekt und führe das Programm aus.
- Ergänze in der Klasse `Husky` beim Attribut `herrchen` deinen Namen und untersuche die Methode `herrchenAusgeben()`.
- Rufe die Methode im Hauptprogramm `TODO1.2` auf. Was fällt dir auf?

Polymorphismus bei höheren Datentypen

Speichert man ein Objekt der _____ in einer Variablen vom Datentyp der _____, so ist es nicht mehr möglich, Attribute und Methoden zu nutzen, die nur auf der Unterklasse definiert sind. Stattdessen wird das Objekt der Unterklasse wie sein Behälter (mit dem Datentyp der Oberklasse) behandelt. Hier hat also eine implizite Typumwandlung stattgefunden. Diese spezielle Art der Typumwandlung, bei der ein Objekt gleichzeitig mehrere Typen hat, bezeichnet man auch als _____ (Vielgestaltigkeit auf Griechisch).



Aufgabe 2) 10_2-3_Hunde

- Ergänze in der Klasse `Hund` nun ebenfalls die Methode `herrchenAusgeben()` und lasse hier "Ein generischer Hund hat leider kein Herrchen" ausgeben.
- Kommentiere die Zeilen 6-21 im Hauptprogramm aus, um diese nicht mehr auszuführen. Kommentiere die Zeilen 27-38 ein, um nun diese auszuführen. Starte das Programm.
- Untersuche die Klassen `Hund`, `Pudel`, `Husky`, `Dackel` und erkläre, wie das Verhalten zustande kommt.

Dynamische Bindung von Objekten

Wird ein Objekt der Unterklasse im Behälter der Oberklasse gespeichert, so sind nur die Methoden der _____ verfügbar. Diese können jedoch von den Unterklassen _____ werden. Aus diesem Grund muss während der Ausführung des Programms geprüft werden, ob die Methode überschrieben ist. Ist das der Fall, so wird die auszuführende Methode der _____ ermittelt und ausgeführt. Diese Art der Vermittlung nennt man auch _____. Um bereits im Programmcode klarzumachen, dass es sich in der Unterklasse um eine überschriebene Methode der Oberklasse handelt, nutzt man die Annotation `@Override`. Diese wird in der Zeile über der entsprechenden Methode eingefügt.